

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student Full Name	Gaziz Kairat
Student Identification Number	230103270
Date & Laboratory Session	18.09.2026
Workstation / Laptop Make & Model	Lenovo
Processor SKU / SoC Model	AMD Ryzen 7 7730U with Radeon Graphics
Core Topology (Physical P-cores / E-cores)	Physical Cores: 8 Logical SMT Threads: 16
Cache Hierarchy Capacity	L1: 512 KB L2: 4 MB L3: 16MB
Host OS & Python Runtime	Windows 11 64-bit (x64) / CPython 3.11.8

■ MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where S(p) is the speedup ratio T(1)/T(p), P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your

terminal: 'python3 task1_amdahl.py'. Record the single-process baseline T(1) and the measured multi-core execution times across all 6 core allocations.

```
import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")
```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	2.3269	1.00x	1.00x	100.0%
p = 2	1.5210	1.67x	2.00x	
p = 4	0.9541	2.66x	4.00x	
p = 8	0.7377	3.43x	8.00x	
p = 12	0.7839	3.23x	12.00x	
p = 16	0.8748	2.90x	16.00x	

[Q1.1] Identify the exact inflection point (core count p*) where your observed speedup curve flattens or

degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

The inflection point is $p^* = 8$ workers. Performance improves up to 8 workers, but adding 12 and 16 workers decreases the observed speedup due to CPU resource contention and multiprocessing overhead. The system reports 16 logical cores, so the workload reaches saturation before all logical cores are efficiently utilized.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Student Response & Architectural Justification:

$$T(1) = 2.5336 \text{ s}$$

$$T(4) = 0.9541 \text{ s}$$

$$S(4) = 2.5336 / 0.9541 = 2.655$$

$$2.655 = 1 / [(1 - P) + P/4]$$

$$0.3766 = 1 - 0.75P$$

$$P = 0.8312$$

$$\text{Parallel fraction} = 83.12\%.$$

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\text{max}} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

Student Response & Architectural Justification:

$$S_{\text{max}} = 1 / (1 - 0.8312) = 5.92x$$

The maximum theoretical speedup is 5.92x. Even with a 128-core server, this workload cannot achieve 128x speedup because approximately 16.88% of the workload is serial. According to Amdahl's Law, the maximum speedup is approximately 5.92x.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements (16 * 8 bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```
import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")
```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier

Adjacent Indices (False Sharing Active)	0.9841	0.9834	0.9829	0.9834	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.9825	0.9819	0.9812	0.9819	1.00x

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

Student Response & Architectural Justification:

In CPython, an integer object is not exactly 8 bytes. A Python integer has an object header, so its memory footprint is larger than a C 64-bit integer. However, the worksheet assumes 8-byte elements for the cache-line calculation.

A cache line is 64 bytes. With a stride of 16 elements:

$$16 \times 8 = 128 \text{ bytes}$$

Since 128 bytes is greater than 64 bytes, adjacent threads access different cache lines. Therefore, an index stride of 16 prevents the threads from accessing the same 64-byte cache line.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification:

Initially, the cache line can be in the Shared state. When Core 0 writes to index 0, it obtains exclusive ownership and changes the cache line to Modified. When Core 1 writes to index 1 on the same cache line, Core 1 sends an invalidation request. Core 0's copy becomes Invalid, and Core 1 obtains the line in the Modified state. Repeated writes cause continuous cache-line invalidations and coherence traffic.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Student Response & Architectural Justification:

Two standard mechanisms for eliminating false sharing are C++11 alignas(64) and Java @Contended. These mechanisms separate frequently modified variables onto different cache lines and reduce cache coherence traffic.

Measured Results

Adjacent Indices: 0.9834 s

Padded Indices: 0.9819 s

Slowdown Factor: 1.00x

The measured slowdown is approximately 1.00x, so this experiment did not show a significant false-sharing performance penalty

on the tested system.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.1765s	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.4578s	2.59x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

1)Exact Corrupted Count: 0 lost updates (Sum reached exactly 2,000,000).

2)Low-Level Explanation:

Normally, an increment operation requires three distinct steps:

LOAD the value from memory into a CPU register.

ADD 1 to the register value.

STORE the register value back into memory.

If two threads interleave these steps, one thread reads stale data before the other writes, leading to lost updates. However, in standard CPython, thread switching is governed by a time-based switch interval (default 5 milliseconds). Because the 500,000 iterations per thread ran very fast, each thread completed within its GIL execution slice without preemption occurring in the middle of the bytecode execution, yielding zero data corruption.

3)Contention Overhead:

The locked counter required 0.4578s versus 0.1765s, representing a 2.59x slowdown. This slowdown is caused entirely by the overhead of 2,000,000 mutex acquire and release operations.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

```
def lockless_worker(ops, results, index):
```

```
    local_acc = 0
```

```
    for _ in range(ops):
```

```
        local_acc += 1
```

```
    results[index] = local_acc
```

```
def run_lockless():
```

```
    ops_per_thread = TOTAL_OPS // NUM_THREADS
```

```
    results = [0] * NUM_THREADS
```

```
    threads = []
```

```
        threading.Thread(
```

```
            target=lockless_worker, args=(ops_per_thread, results, i)
```

```
        )
```

```
        for i in range(NUM_THREADS)
```

```
    ]
```

```
    t0 = time.perf_counter()
```

```
    for t in threads:
```

```
        t.start()
```

```
    for t in threads:
```

```
        t.join()
```

```
    total = sum(results)
```

```
return total, time.perf_counter() - t0
```

Measured Execution Time: ~0.0850s

Speedup Factor over LockedCounter: $0.4578s / 0.0850s = \sim 5.38x$ faster (exceeds the 2.0x requirement).

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

Eliminating shared mutable state using thread-local accumulation and post-reduction completely avoids synchronization overhead. Even advanced primitives like adaptive spinlocks, reader-writer locks, or CAS loops suffer from:

- 1) Cache-line bouncing across CPU cores due to continuous MESI cache invalidations.*
- 2) OS kernel context switching and thread suspension overhead when threads block.*
- 3) Thread serialization, which forces parallel hardware to execute sequentially.*

In contrast, thread-local accumulation keeps all active writes in each core's private L1 cache registers without locks, contention, or cross-core communication until the final single reduction phase.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```

import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()

```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	1.2790	1.00x	1.1433	1.00x
w = 2 Workers	1.1018	0.86x	1.0827	0.95x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]

w = 4 Workers	1.5093	1.18x	2.9759	2.60x
---------------	--------	-------	--------	-------

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

Compute-Bound Behavior:

The compute benchmark has high arithmetic intensity because operations occur inside CPU registers with zero memory bus traffic. Since modern processors have dedicated ALU/FPU execution units per physical core, 2 and 4 workers execute simultaneously without interfering with one another. The execution time stays close to the baseline (1.28s to 1.51s, a modest 1.18x increase due only to process creation and thermal/frequency scaling).

Memory-Bound Degradation:

The memory benchmark creates arrays of 50,000,000 64-bit floats (~400 MB per worker), which drastically exceeds the on-chip CPU cache (L1/L2/L3) and forces continuous reads and writes directly to off-chip DRAM. While CPU cores are replicated, all cores share the same off-chip memory bus and memory controller. At 4 workers (streaming ~1.6 GB concurrently), the DRAM bandwidth completely saturates. Memory requests queue up, memory controller latency spikes, and execution time degrades sharply to 2.98s (a 2.60x slowdown).

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

Typical Host Configuration (Example for DDR4 / DDR5 laptop):

Memory Type: Dual-Channel DDR4-3200 (or DDR5-4800)

Bus Width: 2 channels x 64 bits = 128 bits total (16 bytes per transfer cycle)

Data Rate: 3,200 MT/s (MegaTransfers per second)

Peak Theoretical Bandwidth:

*Bandwidth = 3.2 GT/s * 16 bytes = 51.2 GB/s*

*(For DDR5-4800 Dual-Channel: 4.8 GT/s * 16 bytes = 76.8 GB/s; Apple M-series unified memory ranges from 100 GB/s to 400 GB/s)*

How Bus Width Limits Multi-Core Scalability:

The memory bus width defines the total number of bits transferred per clock cycle between DRAM and the CPU socket. Because the bus width is shared across all physical cores, per-core available bandwidth is equal to Total Bandwidth divided by Active Cores. As more cores run memory-heavy workloads, per-core bandwidth is throttled, hitting the "Memory Wall."

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:

Recommended Choice: CUDA GPU Kernels.

Architectural Justification:

High-dimensional embedding transformations have low arithmetic intensity and are strictly memory-bandwidth bound.

Standard server/desktop CPUs use DDR4/DDR5 channels delivering typically 40 to 80 GB/s (or up to ~200 GB/s on high-end server sockets).

Modern GPUs use specialized wide-bus memory architectures like GDDR6 or HBM3 (e.g., NVIDIA A100/H100/RTX 4090) providing between 1,000 GB/s and 3,350 GB/s of bandwidth (15x to 40x higher than CPUs).

CPU parallelization with OpenMP cannot overcome the physical pin and channel limits of the motherboard DRAM bus. Moving

memory-saturated pipelines to CUDA leverages the massive bus widths and ultra-high memory bandwidth of GPU architectures.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
☐ Task 3 Unsafe Corruption Verified ☐ Task 2 Cache Slowdown Verified ☐ Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType

Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProces sors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator
----------------------------------	---	--

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

count += i * i

return count

Run both experiments with total_work = 50_000_000 split across T

workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- The Empirical Curve:** Plot \$T\$ (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
 - Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*? Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.